SPDM — Software Purpose Definition Module

SPDM — Software Purpose Definition Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Requirements Standard (SRS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Purpose Definition Governance

Module ID: SLGA-SRS-SPDM-001

Origin ID: OOF.CM.SLGA.01.01.01

Version: 1.0

Status: Canonical Draft · Open Standard

Effective Date: Pending Canonical Publication

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Software Purpose Definition Module (SPDM) governs how the intended

purpose of a software system, component, service, or capability is identified, justified, bounded, evaluated, and maintained as a recognizable governance reference for software requirements formation.

SPDM establishes the methodological conditions under which a proposed

software objective may be recognized as a governed software purpose.

It distinguishes an expressed request from an intended outcome, an intended

outcome from its legitimate basis, and a technically achievable capability from a purpose that is appropriate within the applicable governance context.

SPDM applies to software purposes originating from humans, organizations,

AI agents, autonomous systems, automated discovery mechanisms, or combinations of these sources.

Its principal output is:

Governed Software Purpose State

Its foundational distinction is:

Requested Capability ≠ Governed Software Purpose.

Operational Role

The operational role of SPDM is:

To establish an identifiable, justified, context-aware, and bounded

software purpose that can serve as the legitimate reference for subsequent requirements specification without treating the ability to request, generate, or implement software as authority to define its purpose.

SPDM answers the question:

What is this software intended to accomplish, why is that purpose justified, and under what conditions may it govern the formation of software requirements?

SPDM does not determine whether every necessary software requirement has been identified.

It establishes the purpose against which requirements relevance and

completeness can subsequently be assessed.

Position in the SRS Internal Governance Flow

The SRS internal governance flow is:

Software Purpose Definition → Requirements Specification → Software Scope

Management → Requirements Change Control → Requirements Traceability

SPDM governs:

Software Purpose Definition

It is the first Core Module of SRS.

Its principal inputs may include:

• Software objectives and stakeholder requests

• Business or operational needs

• Existing software purpose records

• Relevant organizational mandates

• Applicable external obligations

• Operational observations

• Proposed software capabilities

• Governed evidence from competent sources

Its principal output is:

Governed Software Purpose State

That state proceeds to:

RSPM — Requirements Specification Module

Central Operational Question

What legitimate outcome is the software intended to support, which software object and operating context does that purpose concern, and is the purpose sufficiently justified, bounded, and explicit to serve as a requirements-governance reference?

Software Purpose Governance Object

Every SPDM assessment must identify the Software Purpose Governance Object.

The object represents the software system, component, service, function, or

proposed capability whose intended purpose is being established.

It should identify, where applicable:

• Software object identity

• Intended capability

• Relevant organizational or operational context

• Purpose origin

• Applicable stakeholders or beneficiaries

• Existing software relationships

• Purpose version or reference state

A purpose may apply to an entire software system or to a sufficiently identifiable subordinate capability.

Software Object Identity ≠ Software Purpose.

The existence of software does not establish why it should exist or what it is legitimately intended to accomplish.

Purpose Origin and Request Identification

SPDM shall identify the origin of a proposed software purpose.

Possible origins include:

• Human instructions

• Organizational initiatives

• Business requirements

• Contractual commitments

• Applicable external obligations

• Operational needs

• Identified deficiencies in existing software

• AI-generated proposals

• Autonomous system observations

The origin should remain traceable.

However, the source of a request does not independently establish the legitimacy of the proposed purpose.

Purpose Origin ≠ Purpose Legitimacy.

Where the origin is unknown or insufficiently supported, the uncertainty shall remain explicit.

Purpose Statement and Intended Outcome

A governed software purpose shall express the outcome the software is intended to support.

The purpose statement should be sufficiently clear to distinguish:

The problem or need: Why the software capability is being considered.

The intended outcome: What legitimate result the software is expected to enable.

The software contribution: How the software is expected to contribute to that outcome.

The relevant context: Where and for whom the outcome is intended.

The governing conditions: Material restrictions or assumptions affecting

the purpose.

A purpose should not be defined solely by a preferred technology, implementation method, or feature list.

For example, “deploy an AI agent” describes a proposed technical approach.

It does not, by itself, establish the operational purpose the agent is intended to serve.

Technical Solution ≠ Software Purpose.

Purpose Legitimacy and Authority Boundary

SPDM shall distinguish the existence of a proposed purpose from its

legitimate recognition.

A proposed purpose may require confirmation of:

• The applicable organizational mandate

• The competence of the requesting actor

• The permitted software function

• Relevant external constraints

• The boundaries of delegated authority

• Conditions established by competent governance processes

SPDM does not independently create or grant authority.

Where competent authority cannot be established, the purpose shall not be represented as unconditionally authorized.

Requested Purpose ≠ Authorized Purpose.

Technical Capability ≠ Legitimate Purpose.

An AI agent may identify an operational problem and propose a software solution without possessing authority to redefine the organization's software objectives.

Purpose Relevance and Justification

SPDM shall evaluate whether the proposed purpose is meaningfully connected to an identifiable need, obligation, or intended outcome.

The assessment should consider:

• The problem the software is intended to address

• The expected contribution of the software

• The relevance of the proposed capability

• Available evidence supporting the need

• Whether the proposed purpose duplicates, conflicts with, or materially

• Whether the justification depends on unresolved assumptions

expands an existing purpose.

SPDM does not require every software purpose to demonstrate immediate financial return.

Legitimate purposes may concern safety, accessibility, continuity, compliance, research, public benefit, operational capability, or other justified outcomes.

A technically feasible purpose may nevertheless lack a sufficient governance basis.

Feasibility ≠ Justification.

Purpose Boundary and Intended Use

SPDM shall identify the material boundaries of the software purpose.

These may include:

• Intended users or operating actors

• Intended functions

• Intended operating environments

• Material excluded uses

• Applicable organizational boundaries

• Contextual or jurisdictional limitations

• Relevant operating conditions

• Prohibited or unauthorized purpose extensions

Purpose boundaries shall distinguish what the software is intended to accomplish from additional functions it may technically be capable of performing.

A software capability shall not automatically acquire a broader legitimate

purpose merely because its technical functionality permits broader use.

Available Capability ≠ Authorized Intended Use.

Detailed requirements applicability and scope management remain governed by

the subsequent SRS Core Modules.

Purpose–Operational Reality Alignment

A software purpose may depend on operational conditions that are not yet established.

SPDM shall identify material assumptions concerning:

• The operating environment

• Available infrastructure

• Relevant external systems

• Human or organizational capabilities

• Data availability

• Resource limitations

• Operational dependencies

• Conditions necessary for the intended outcome

SPDM does not independently certify operational feasibility.

Where the intended purpose depends on unverified conditions, those assumptions shall remain identifiable for subsequent requirements specification and competent operational reality validation.

Intended Outcome ≠ Operationally Achievable Outcome.

Purpose Coherence and Conflict

SPDM shall identify material contradictions within a proposed purpose or between that purpose and other applicable governed purposes.

Examples include:

- A purpose requiring unrestricted data access while another applicable

constraint prohibits that access. - A software objective that conflicts with an established organizational mandate. - An autonomous agent expanding its intended purpose beyond its delegated task. - A proposed capability whose intended use contradicts a non-negotiable external constraint.

Conflicts shall not be resolved merely by combining incompatible purpose

statements.

Where the competent basis for resolution is unavailable, the purpose state shall remain conditional, conflicted, or unresolved.

Purpose Aggregation ≠ Purpose Coherence.

Conflict Identification ≠ Authority to Resolve Conflict.

Purpose Recognition and Governance States

SPDM shall distinguish purpose evaluation from purpose recognition.

The canonical purpose recognition sequence is:

Proposed Purpose → Purpose Evaluation → Purpose Recognition → Governed

Software Purpose State

Possible governance states include:

Proposed: The purpose has been submitted or generated but has not

completed applicable evaluation.

Under Evaluation: The purpose is being assessed for legitimacy, clarity, justification, boundaries, and relevant conditions.

Recognized: The purpose has satisfied the applicable recognition

conditions.

Conditionally Recognized: The purpose may serve as a governed reference subject to explicit conditions.

Conflicted: Material purpose contradictions remain unresolved.

Evidence-Limited: Material evidence required for recognition is insufficient.

Unresolved: The available information does not support a sufficiently determined purpose state.

Superseded: A later governed purpose state has replaced the previous

state.

Withdrawn: The purpose has been withdrawn through an applicable governance process.

Purpose recognition shall preserve its relevant conditions and evidence

basis.

Evaluated Purpose ≠ Recognized Purpose.

Recognized Purpose ≠ Authorization to Implement Software.

Purpose Change and Continuity

Software purpose may change during the lifecycle.

Changes may arise from:

• New business or operational needs

• Organizational restructuring

• Changes in applicable obligations

• Software modernization

• Changes in intended users

• New operating environments

• Autonomous system proposals

• New evidence concerning the original purpose

• Discovery that the intended outcome is no longer relevant

SPDM shall distinguish a clarification of an existing purpose from a

material change to that purpose.

Material changes should preserve:

• Previous purpose identity

• Proposed new purpose

• Change origin

• Change justification

• Relevant authority conditions

• Affected purpose boundaries

• Recognition state

• Effective context

• Implications for existing requirements

A material purpose change may require reassessment of the governed requirements baseline through SRS.

Purpose Change ≠ Automatically Authorized Requirements Change.

Purpose continuity shall remain reconstructable across changes in software

creators, organizations, AI agents, technologies, and documentation systems.

Purpose Evidence, Uncertainty, and Traceability

SPDM shall preserve sufficient evidence to support the established purpose

governance state.

Material evidence may include:

• Original requests

• Organizational mandates

• Applicable external obligations

• Operational problem descriptions

• Purpose justification records

• Recognition records

• Relevant authority references

• Material assumptions

• Purpose change history

The canonical traceability relationship is:

Purpose Origin → Purpose Statement → Justification → Authority Conditions

→ Purpose Boundary → Recognition State → Governed Software Purpose State

Evidence shall not be represented as more complete or authoritative than

its source supports.

Where material evidence is missing, conflicting, outdated, or insufficiently reliable, the limitation shall remain explicit.

Documented Purpose ≠ Evidenced Purpose.

Evidence of Request ≠ Evidence of Authorization.

Purpose Reassessment

A Governed Software Purpose State shall remain capable of reassessment.

Relevant triggers may include:

• Material changes in intended use

• Loss or change of the underlying organizational mandate

• New applicable constraints

• Material operational assumption failure

• Discovery of purpose conflicts

• Material changes in the software object

• Evidence that the intended outcome is no longer justified

• Autonomous reinterpretation or expansion of the purpose

• Requirements findings that reveal an inadequately defined purpose

Reassessment shall determine whether the existing purpose remains

recognized, requires modification, becomes conditional, or must be withdrawn or superseded.

Historical Purpose Recognition ≠ Permanent Purpose Validity.

Governed Software Purpose State

The principal output of SPDM is: Governed Software Purpose State

The state shall preserve sufficient information to identify:

Software Purpose Governance Object: The software object or capability

concerned.

Purpose Identity: The identifiable purpose and relevant version.

Purpose Statement: The intended outcome and software contribution.

Purpose Origin: The source or recognized basis of the proposed purpose.

Purpose Justification: The evidence-supported reason for establishing the

purpose.

Authority Boundary: Applicable recognition and authorization conditions.

Purpose Boundary: Intended use, material exclusions, and contextual

limitations.

Operational Assumptions: Conditions upon which the intended outcome

materially depends.

Purpose Coherence: Material conflicts or unresolved contradictions.

Recognition State: The applicable purpose governance status.

Evidence State: Supporting evidence and material limitations.

Uncertainty: Material unresolved conditions.

Continuity: Relevant history and superseded purpose states.

Traceability: Relationships to the origin, recognition basis, and

subsequent requirements formation.

The state shall be sufficiently explicit for use by the Requirements

Specification Module.

It shall not imply that the software has been implemented, validated, or authorized for deployment.

Minimum Implementation Framework

A conforming implementation of SPDM must provide the following six functional capabilities.

• Software Purpose Object and Origin Governance

Capability to identify the governed software object, proposed purpose,

purpose origin, intended outcome, and relevant operating context.

The implementation must preserve the distinction between a software object, a requested capability, and a proposed purpose.

• Purpose Justification and Authority Boundary Governance

Capability to evaluate the proposed purpose against its identified need,

supporting evidence, applicable mandate, and competent recognition conditions.

The implementation must not convert a request or technical capability into independent governance authority.

• Purpose Definition and Boundary Governance

Capability to establish a sufficiently clear purpose statement, intended

outcome, software contribution, intended use, material exclusions, and contextual limitations.

The implementation must identify material ambiguity and conflicting purpose conditions.

• Operational Assumption and Evidence Governance

Capability to identify material operational assumptions, preserve

supporting evidence, assess evidence limitations, and flag conditions requiring competent feasibility validation.

The implementation must distinguish assumed conditions from verified operational reality.

• Purpose Recognition, Change, and Continuity Governance

Capability to establish purpose governance states, preserve recognition

conditions, identify material purpose changes, maintain relevant versions, and trigger reassessment when governing conditions change.

The implementation must preserve purpose history without treating previous recognition as permanently valid.

• Governed Purpose State and Requirements Handoff Governance

Capability to produce an identifiable, traceable Governed Software Purpose

State containing the information necessary for subsequent requirements specification.

The implementation must preserve unresolved conditions, evidence limitations, and authority boundaries during handoff.

GIE™ Compatibility

SPDM is compatible with GIE™ as a downstream methodological execution and analysis layer.

A GIE™ implementation may support purpose discovery, structured assessment, conflict detection, evidence mapping, state representation, reassessment, and requirements handoff.

GIE™ must preserve the canonical SPDM governance distinctions, recognition conditions, and authority boundaries.

It must not independently transform a proposed purpose into an authorized purpose unless the applicable competent recognition process has established that state.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical SPDM governance logic is:

Identify Software Purpose Governance Object

↓

Identify Proposed Purpose and Origin

↓

Establish Intended Outcome and Software Contribution

↓

Identify Purpose Context and Relevant Stakeholders

↓

Assess Purpose Relevance and Justification

↓

Identify Applicable Authority and Recognition Conditions

↓

Define Intended Use and Purpose Boundaries

↓

Identify Material Operational Assumptions

↓

Assess Purpose Clarity and Coherence

↓

Identify Conflicts and Unresolved Conditions

↓

Assess Evidence Sufficiency and Uncertainty

↓

Determine Purpose Recognition State

↓

Recognized / Conditionally Recognized / Conflicted / Evidence-Limited /

Unresolved

↓

Establish Governed Software Purpose State

↓

Preserve Purpose Identity, Evidence, and Traceability

↓

Handoff to Requirements Specification Module

↓

Reassess When Material Purpose Conditions Change

The principal output is:

Governed Software Purpose State

Governance Outputs

SPDM may produce:

- Software Purpose Governance Object

- Purpose Origin Record - Purpose Statement - Intended Outcome Record - Purpose Justification Record - Purpose Context Record - Purpose Boundary Record - Intended Use Record - Purpose Authority Boundary Record - Purpose Operational Assumption Record - Purpose Coherence Assessment - Purpose Conflict Record - Purpose Evidence State - Purpose Uncertainty Record - Purpose Recognition State - Purpose Recognition Conditions - Purpose Change Record - Purpose Reassessment Record - Purpose Continuity Record - Purpose Traceability Record - Requirements Handoff Record

The principal output is:

Governed Software Purpose State

Critical Distinctions

SPDM preserves the following distinctions:

Requested Capability ≠ Governed Software Purpose

Software Object Identity ≠ Software Purpose

Purpose Origin ≠ Purpose Legitimacy

Requested Purpose ≠ Authorized Purpose

Technical Solution ≠ Software Purpose

Technical Capability ≠ Legitimate Purpose

Feasibility ≠ Justification

Available Capability ≠ Authorized Intended Use

Intended Outcome ≠ Operationally Achievable Outcome

Operational Assumption ≠ Verified Operational Condition

Purpose Aggregation ≠ Purpose Coherence

Conflict Identification ≠ Authority to Resolve Conflict

Evaluated Purpose ≠ Recognized Purpose

Recognized Purpose ≠ Authorization to Implement Software

Documented Purpose ≠ Evidenced Purpose

Evidence of Request ≠ Evidence of Authorization

Purpose Change ≠ Automatically Authorized Requirements Change

Historical Purpose Recognition ≠ Permanent Purpose Validity

Purpose Recognition ≠ Requirements Completeness

Purpose Definition ≠ Software Implementation

Cross-Architecture Boundary

SPDM governs software purpose definition within SRS and SLGA™.

It does not replace the governance domains responsible for general authority, accountability, autonomous system behavior, operational reality, validation, or evidence integrity.

Relevant interoperating architectures may include:

GOA™ — Governance Architecture: General authority, mandate, and

delegation boundaries.

AGA™ — Accountability Governance Architecture: Applicable responsibility

and accountability conditions.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous system

authority and execution boundaries.

ORA™ — Operational Reality Architecture: Competent assessment of

operational reality conditions.

VALIDOS™: Applicable validation methodology.

ADIT®: Broader evidence and auditability governance.

Within SRS:

SPDM governs software purpose definition.

RSPM governs requirements specification and completeness assessment.

SSMM governs detailed requirements scope and applicability.

RCCM governs requirements changes.

RTM governs requirements traceability.

SPDM provides the purpose reference for these functions without assuming

their governance responsibilities.

Canonical Boundary

SPDM begins when a proposed software objective, need, requested capability, or intended outcome requires recognition as a governed software purpose.

SPDM governs:

• Software purpose object identification

• Purpose origin

• Purpose statement formation

• Intended outcome identification

• Purpose relevance and justification

• Applicable authority and recognition boundaries

• Intended use and purpose boundaries

• Material operational assumptions

• Purpose coherence and conflict

• Purpose evidence and uncertainty

• Purpose recognition

• Purpose governance states

• Purpose changes

• Purpose reassessment

• Purpose continuity

• Purpose traceability

• Requirements handoff

SPDM does not independently determine:

• Complete software requirements

• Detailed requirements applicability

• Software design

• Software implementation

• Software quality or security conformance

• Operational feasibility certification

• Release or deployment authorization

• General organizational or legal authority

SPDM ends when the software purpose has been represented as a:

Governed Software Purpose State

with sufficient identification, justification, boundaries, recognition

conditions, evidence, and traceability to support subsequent requirements specification.

Use Case 1 — The Organization Wants an AI Agent but Has Not Defined Its

Purpose

A company decides to introduce an AI agent into its customer service

operations.

Management requests:

“Build an AI agent that can handle customer requests automatically.”

The development team can begin designing the agent.

However, the request does not establish:

• Which customer problems the agent should resolve

• Which outcomes are legitimate

• Whether the agent may modify customer accounts

• Which actions require human intervention

• Which functions remain outside its intended purpose

• What operational conditions the agent depends on

SPDM identifies that the requested capability is not yet a sufficiently governed software purpose.

The organization must establish the intended outcome, relevant boundaries, and competent recognition conditions.

The resulting state may become:

Conditionally Recognized

until the material purpose conditions are resolved.

Once sufficiently established, SPDM produces a:

Governed Software Purpose State

that can be used by RSPM to form the applicable software requirements.

The ability to build an AI agent does not establish what the agent is legitimately intended to do.

Use Case 2 — A Software Feature Expands Beyond Its Original Purpose

An enterprise application was originally created to generate operational

performance reports.

Over time, additional functionality allows it to combine employee activity

records, access logs, and productivity information.

A development team proposes using the same software to automatically rank

individual employees.

The software may technically support this function.

However, the proposed use materially differs from the original reporting

purpose.

SPDM identifies the change as a potential purpose expansion rather than an

ordinary implementation detail.

The assessment considers:

• The original recognized purpose

• The proposed new purpose

• The origin of the expansion request

• Applicable organizational authority

• Relevant intended-use boundaries

• Material external constraints

• Whether a new purpose recognition process is required

The resulting state may remain:

Under Evaluation

or:

Unresolved

until the applicable governance conditions are established.

The software's existing technical capability does not authorize the new purpose.

Functional Expansion ≠ Legitimate Purpose Expansion.

Canonical Closing Statement

Software can be created before its purpose has been properly established.

A developer can implement a requested feature.

An AI coding agent can transform an instruction into working software.

An autonomous system can identify a problem and generate a technical solution.

None of these capabilities independently establishes that the resulting software purpose is legitimate, sufficiently defined, or appropriate for its intended operating context.

Requested Capability ≠ Governed Software Purpose.

Technical Solution ≠ Software Purpose.

Purpose Origin ≠ Purpose Legitimacy.

Recognized Purpose ≠ Authorization to Implement Software.

A software purpose must remain connected to an identifiable outcome, a justified basis, applicable boundaries, and the conditions under which it has been recognized.

It must remain distinguishable from the technical means used to achieve it.

And it must remain traceable when software, organizations, development methods, or autonomous actors change.

SPDM does not determine how the software must be built.

It establishes what the software is legitimately intended to accomplish before that purpose becomes the basis for requirements formation.

SPDM transforms proposed software objectives into governed purpose states

— ensuring that the ability to create software never substitutes for the legitimacy, clarity, and continuity of the purpose it is intended to serve.